

Documento técnico CI/CD DevOps

9 AGOSTO

Universidad de la sabana

Creado por: Juan Sebastian Buitrago Romero



Documentación Técnica – Pipeline CI/CD con GitHub Actions y Jenkins

Introducción

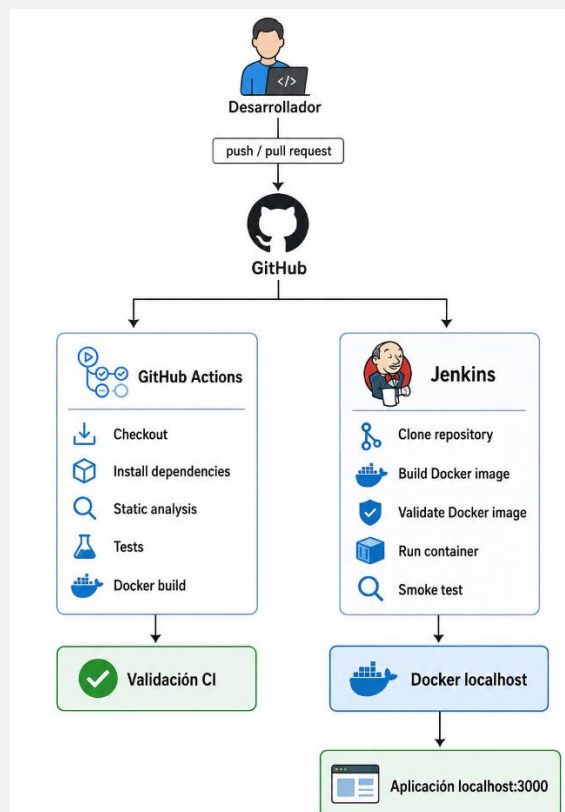
El objetivo de este laboratorio es implementar un flujo básico de Integración Continua (CI) y Entrega Continua (CD) para una aplicación web almacenada en un repositorio GitHub.

Para ello se utilizan dos herramientas:

- GitHub Actions, encargado del proceso de Integración Continua.
- Jenkins, encargado de definir y ejecutar el proceso de construcción y validación del artefacto Docker.

La aplicación utilizada en el laboratorio está desarrollada con Node.js y Express y dispone de pruebas automatizadas mediante Jest y análisis estático mediante ESLint.

La solución implementada sigue el siguiente flujo



1. Aplicación utilizada

Para realizar las pruebas del pipeline se ha creado una aplicación web sencilla utilizando Node.js y Express.

La aplicación expone principalmente dos endpoints:

- GET /
- GET /health

El endpoint /health permite comprobar automáticamente si la aplicación se encuentra funcionando correctamente.

Una respuesta correcta es:

```
{
  "status": "UP"
}
```

Este endpoint resulta especialmente útil para realizar el Smoke Test desde Jenkins después de arrancar el contenedor Docker.

2. Integración Continua con GitHub Actions

GitHub Actions se utiliza como herramienta de Continuous Integration (CI).

Su responsabilidad consiste en validar automáticamente cualquier modificación realizada sobre el código antes de que pueda continuar hacia las siguientes fases del ciclo de entrega.

La configuración se encuentra en:

`.github/workflows/ci.yml`

3. Activación del workflow

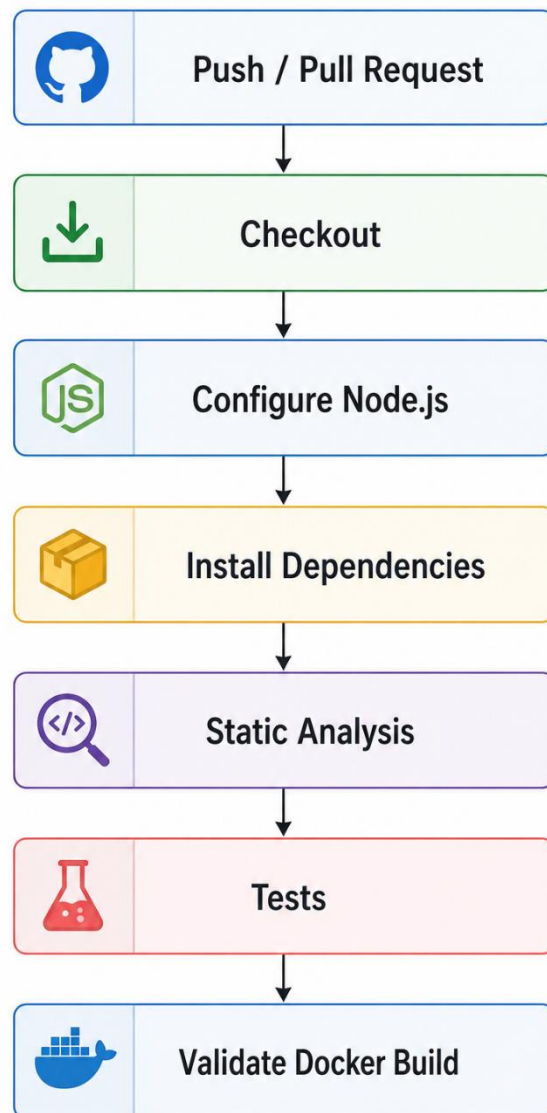
El pipeline se configura para ejecutarse automáticamente cuando se produce un:

- push
un:
- pull_request

sobre las ramas:

- main
- develop

Esto permite que cualquier cambio enviado al repositorio sea validado automáticamente.
El flujo CI implementado es:



4. Steps de GitHub Actions

4.1. Checkout source code

El primer step obtiene el código fuente del repositorio GitHub.

- name: Checkout source code
uses: actions/checkout@v4

La acción `actions/checkout` descarga el contenido del repositorio dentro del runner temporal utilizado por GitHub Actions.

Este paso es necesario porque los siguientes comandos necesitan acceder al código fuente, al `package.json`, al `Dockerfile` y a las pruebas.

4.2. Configure Node.js

El siguiente step configura Node.js dentro del runner:

```
- name: Configure Node.js
  uses: actions/setup-node@v4
  with:
    node-version: "20"
```

En este laboratorio se utiliza Node.js 20.

Este step garantiza que el entorno utilizado por GitHub Actions dispone de una versión conocida y reproducible de Node.js.

4.3. Install dependencies

Una vez configurado Node.js se instalan las dependencias de la aplicación:

```
- name: Install dependencies
  run: npm install
```

El comando consulta el archivo:

- package.json

e instala las dependencias necesarias.

Entre ellas se encuentran:

- Express.
- Jest.
- Supertest.
- ESLint.

Esta fase prepara el entorno necesario para poder ejecutar posteriormente el análisis estático y las pruebas.

4.4. Static code analysis

El siguiente step realiza un análisis estático del código mediante ESLint:

```
- name: Static code analysis
  run: npm run lint
```

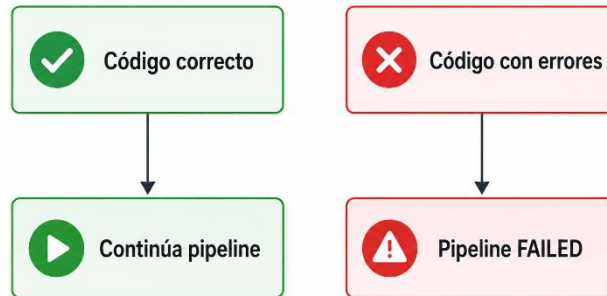
Este comando ejecuta el script definido en **package.json**.

El objetivo del análisis estático es detectar problemas como:

- Variables no definidas.
- Variables declaradas pero no utilizadas.
- Errores básicos de calidad del código.

- Incumplimientos de las reglas configuradas en ESLint.

Si ESLint encuentra un error configurado como bloqueante, el comando devuelve un código de salida diferente de cero y GitHub Actions detiene el pipeline.



5. Ejecución de pruebas

El siguiente step ejecuta las pruebas automatizadas:

```
- name: Run tests
  run: npm test
```

Para este laboratorio se utiliza Jest como framework de pruebas y Supertest para realizar peticiones HTTP sobre la aplicación Express.

Entre otras comprobaciones, las pruebas verifican que:

GET /

devuelve un código HTTP:

- 200

y que:

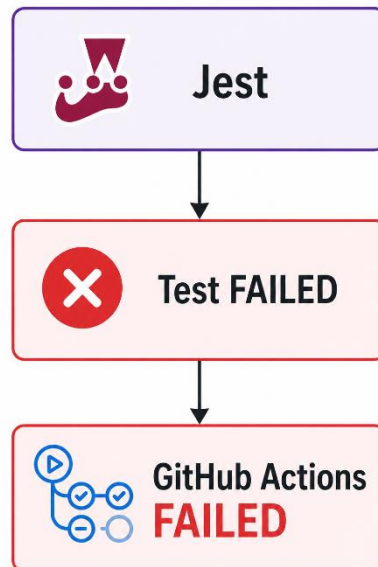
GET /health

devuelve:

```
{
  "status": "UP"
}
```

Este step permite comprobar que los cambios realizados en el código no han roto el comportamiento básico de la aplicación.

Si una prueba falla:



De esta manera se evita continuar con una versión del código que no supera las pruebas definidas.

6. Validación de la imagen Docker

El último step del proceso CI valida que la aplicación pueda empaquetarse correctamente como una imagen Docker.

- name: Validate Docker build

run: docker build -t cicc-lab-webapp:\${{ github.sha }} .

Docker utiliza el archivo:

- Dockerfile

Para construir la imagen.

Como tag se utiliza el identificador del commit:

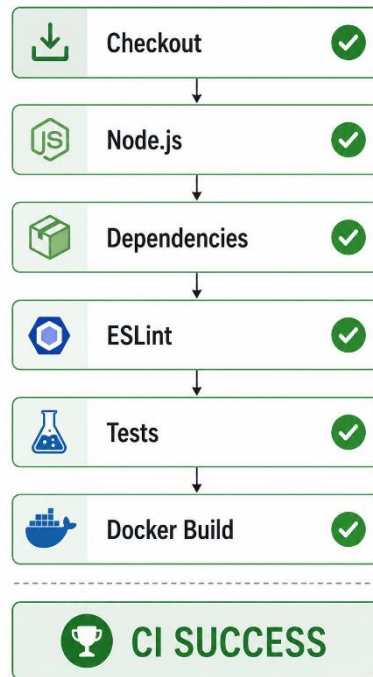
- github.sha

Conceptualmente se obtiene una imagen como:

- cicc-lab-webapp:a821df...

Este paso es importante porque comprueba que el código no solamente supera las pruebas, sino que también puede convertirse correctamente en un artefacto desplegable.

El workflow de GitHub Actions termina entonces con:



7. Pipeline de Jenkins

Jenkins se utiliza para implementar la segunda parte del laboratorio.

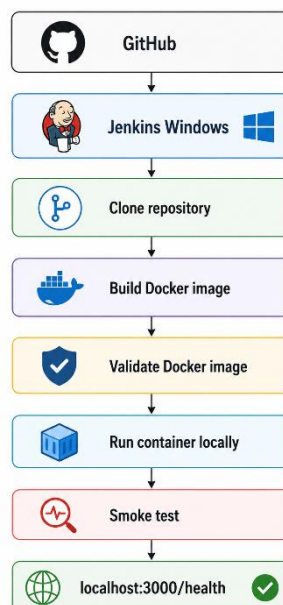
La definición del pipeline se almacena dentro del repositorio mediante el archivo:

- Jenkinsfile

Esto permite aplicar el concepto de Pipeline as Code, ya que la definición del proceso de automatización se versiona junto con el código fuente.

En este laboratorio Jenkins se encuentra instalado directamente sobre Windows y utiliza el Docker Engine disponible en la máquina local.

El flujo implementado es:



8. Stage – Clone repository

El primer stage de Jenkins obtiene el código fuente desde GitHub.

Conceptualmente:

```
stage('Clone repository') {  
    steps {  
        git branch: "${params.GIT_BRANCH}",  
        url: "${params.GIT_REPOSITORY}"  
    }  
}
```

El repositorio y la rama se reciben como parámetros.

Por ejemplo:

- GIT_REPOSITORY = <https://github.com/usuario/cicd-lab.git>
- GIT_BRANCH = main

Esto permite reutilizar el mismo Jenkinsfile sin tener que modificar directamente su código.

Después de este stage, Jenkins dispone dentro de su workspace de:

- src/
- test/
- Dockerfile
- package.json
- Jenkinsfile
- .github/

9. Stage – Build Docker image

El siguiente stage construye la imagen Docker de la aplicación.

Jenkins utiliza el número de ejecución:

- BUILD_NUMBER

para generar una versión de la imagen.

Por ejemplo, para el build número 15:

- cicd-lab-webapp:15

El comando ejecutado es conceptualmente:

- `docker build -t cicd-lab-webapp:15 .`

También se genera un tag:

- latest

obteniendo:

- `cicd-lab-webapp:15`
- `cicd-lab-webapp:latest`

Esto permite conservar una referencia a la versión concreta del build y otra referencia sencilla a la última versión construida.

La imagen permanece almacenada en el **Docker Engine local de Windows**.

10. Stage – Validate Docker image

Después de construir la imagen se comprueba que Docker la haya generado correctamente.

Se utiliza:

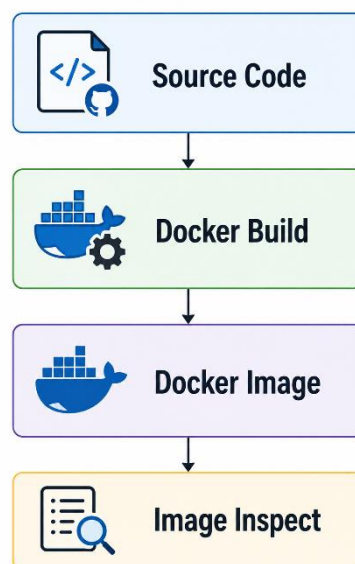
- `docker image inspect`

sobre las imágenes creadas.

Por ejemplo:

- `docker image inspect cicd-lab-webapp:15`
- `docker image inspect cicd-lab-webapp:latest`

Este stage permite detectar inmediatamente un problema si la imagen esperada no existe. El flujo hasta este punto es:



11. Stage – Run container locally

Una vez validada la imagen, Jenkins ejecuta un contenedor utilizando la imagen recién construida.

Antes de arrancarlo se elimina cualquier contenedor anterior utilizado por el laboratorio para evitar conflictos de nombres o puertos.

Posteriormente se ejecuta:

```
docker run -d ^  
--name cicd-lab-webapp-test ^  
-p 3000:3000 ^  
%FULL_IMAGE%
```

El parámetro:

-d

ejecuta el contenedor en segundo plano.

El parámetro:

--name cicd-lab-webapp-test

asigna un nombre conocido al contenedor.

Finalmente:

-p 3000:3000

publica el puerto 3000 del contenedor sobre el puerto 3000 de Windows.

La aplicación queda temporalmente disponible en:

<http://localhost:3000>

12. Espera para el arranque de la aplicación

Después de ejecutar el contenedor se realiza una pequeña espera para permitir que Node.js termine de arrancar.

Inicialmente se utilizó el comando de Windows:

timeout /t 5 /nobreak

Sin embargo, durante la ejecución desde Jenkins se obtuvo:

ERROR: No es compatible la redirección de entradas

Esto ocurre porque Jenkins ejecuta los comandos batch con una gestión de entrada estándar diferente de una consola interactiva de Windows.

Para solucionar el problema se substituyó timeout por PowerShell:

powershell -Command "Start-Sleep -Seconds 5"

De esta forma Jenkins puede esperar unos segundos sin depender de una entrada interactiva.

13. Stage – Smoke Test

Una vez iniciado el contenedor se realiza una comprobación funcional mínima o Smoke Test.

Se ejecuta:

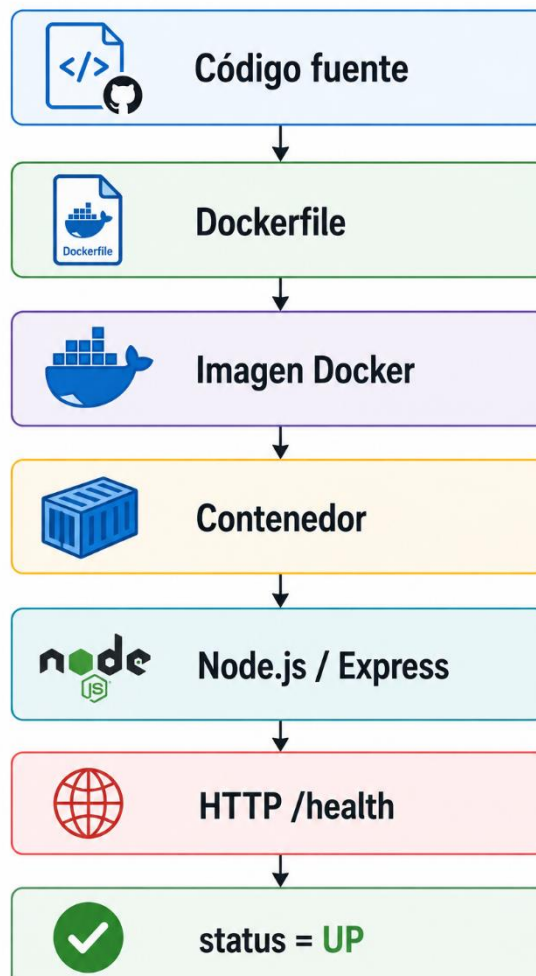
- `curl --fail http://localhost:3000/health`

El objetivo es verificar que no solamente existe la imagen y que el contenedor se ha iniciado, sino que la aplicación realmente responde mediante HTTP.

La respuesta esperada es:

```
{  
  "status": "UP"  
}
```

Por tanto, este stage valida la cadena completa:



Si curl no puede conectarse o recibe una respuesta HTTP de error, el comando devuelve un código de salida de error y Jenkins marca el stage como fallido.

14. Post Actions de Jenkins

El pipeline dispone también de acciones posteriores a la ejecución.
Independientemente del resultado, se elimina el contenedor temporal:
cicd-lab-webapp-test

Esto evita dejar contenedores ejecutándose después de finalizar el laboratorio.
La imagen Docker, sin embargo, permanece disponible localmente.
Puede comprobarse mediante:
docker images

En caso de ejecución correcta Jenkins informa de que el pipeline ha terminado satisfactoriamente.
Si alguno de los stages falla, Jenkins marca la ejecución como:

- FAILED

Permitiendo revisar los logs del stage correspondiente.

15. Configuración de Docker en Jenkins sobre Windows

Durante la configuración se detectó que los comandos Docker funcionaban correctamente desde CMD/PowerShell:

- docker --version
- docker info

pero inicialmente Jenkins mostraba:

- "docker" no se reconoce como un comando interno o externo

La causa era que el entorno utilizado por el servicio de Jenkins no tenía disponible la ruta del ejecutable de Docker.

Docker Desktop instala normalmente el cliente en una ubicación similar a:

- C:\Program Files\Docker\Docker\resources\bin

Una vez incorporada correctamente esta ubicación al entorno utilizado por Jenkins y reiniciado el servicio, Jenkins pudo ejecutar:

- docker --version
- docker build
- docker image inspect
- docker run
- docker ps

Este punto demuestra una consideración importante de los pipelines: las herramientas disponibles para el usuario interactivo de Windows no necesariamente están disponibles automáticamente para el servicio de Jenkins.

16. Separación de responsabilidades entre GitHub Actions y Jenkins

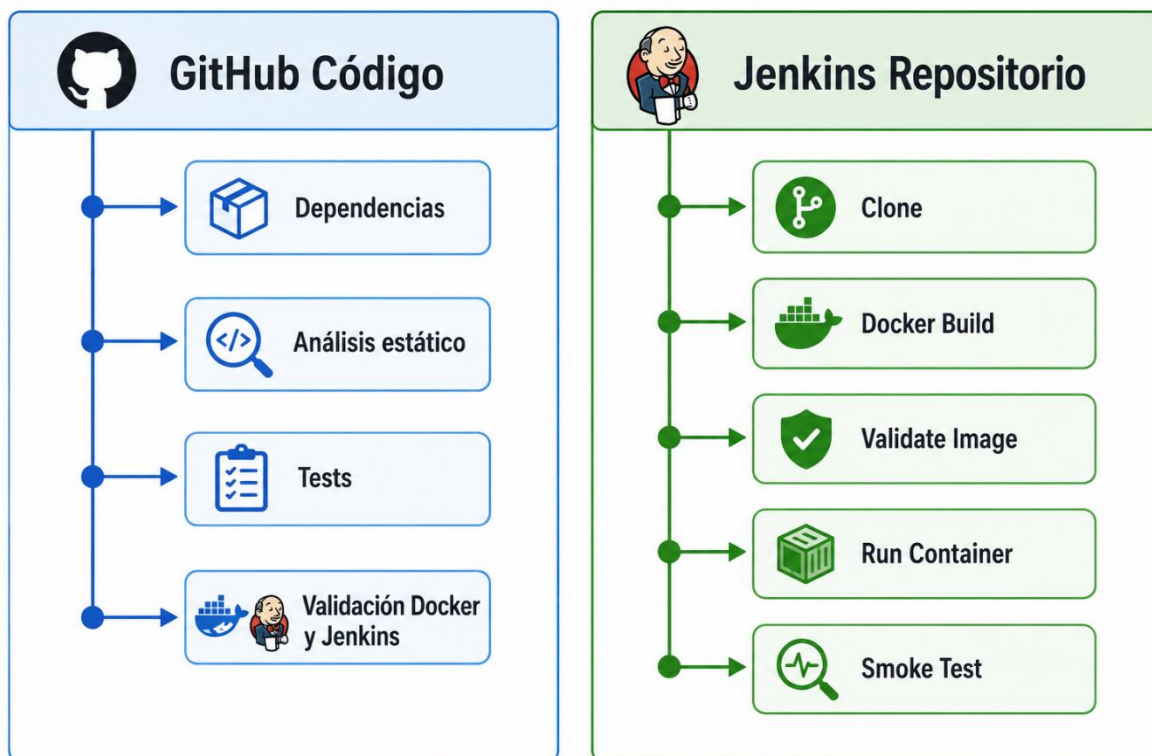
La solución divide las responsabilidades del proceso CI/CD.

GitHub Actions – CI

GitHub Actions se ocupa principalmente de validar el código

Jenkins – CD

Jenkins se ocupa de generar y comprobar el artefacto ejecutable



Su objetivo en el laboratorio es demostrar que la versión puede empaquetarse y ejecutarse correctamente.